CUDA C

Compute Unified Device Architecture

Outline

- **GPU Computing**
- CUDA C programming

GPU Computing and CUDA C programming

- GPU Computing and CUDA (Compute Unified Device Architecture) C programming
  - Assumes that the CUDA threads execute on a physically separate ***device*** that operates as a coprocessor to the ***host*** running the C program.
  - The **kernels** execute on a **GPU** and the rest of the C program executes on a **CPU**.
  - Both the ***host*** and the ***device*** maintain their own separate memory spaces, referred to as ***host memory*** and ***device memory***

What is GPU ?

- GPU : Graphics Processing unit
  - Highly parallel specialized circuit for computer graphics.

NVIDIA GeForce RTX 3080



AMD Radeon HD 6870



  - Game consoles, WS/Servers, Smartphones, PC, embedded systems, …

GFLOPS

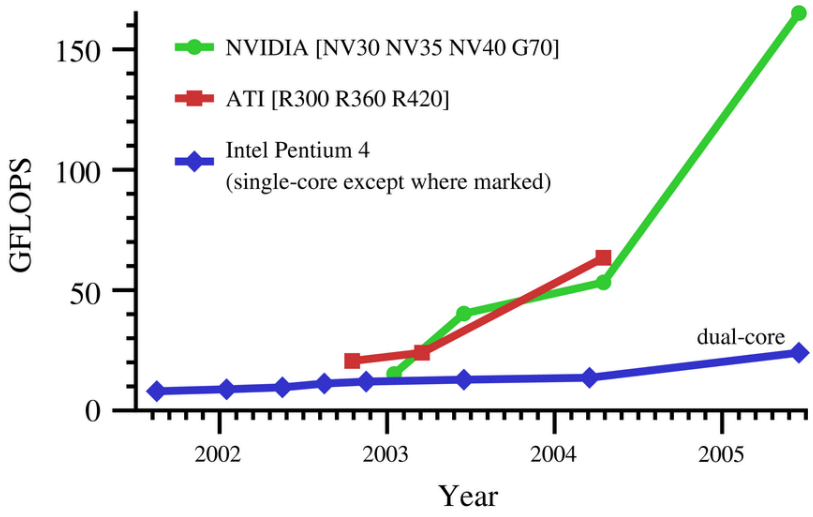
- FLOPS = flop/s, for floating-point operations per second
- a measure of a computer's performance
- especially in fields of scientific calculations that make heavy use of floating-point calculations
- MIPS = X instructions per second
  - measure of CPU instruction execution speed
  - (number of cycles per second /cycles per instruction)
- gigaFLOPS

number of cycles per second = frequency of the processor
cycles per instruction= clock cycles to execute a single instruction

A bit of history

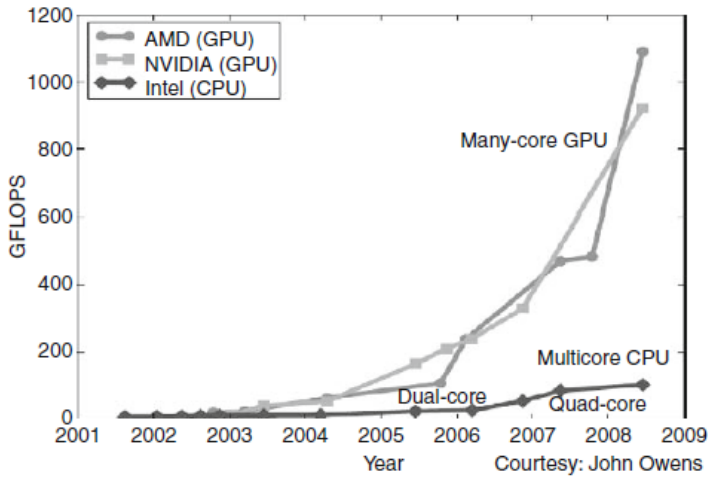
- Sony, IBM and Toshiba launched in 2006 the Cell processor
  - a dozen PS3 consoles can be attached together to form the equivalent of a supercomputer.
- Similarly, GPU GRAPHICS Processors for PC video games, have been increasingly used since NVIDIA released CUDA in 2007

The start: out of PlayStation PS2 and PS3, and PC Video Games !



Borrowed from GPGPU Workshop slides (SIGGRAPH 05)

1 teraflops (1000 gigaflops) versus 100 gigaflops in 2009.



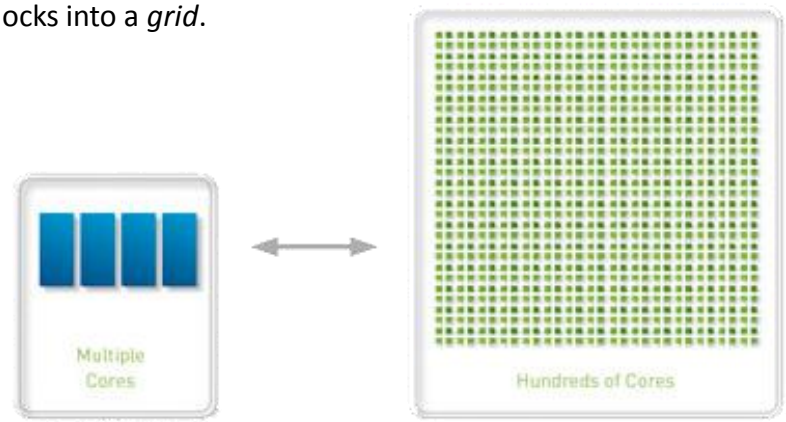
Enlarging performance gap between GPUs and CPUs.

What is GPU computing ?

- Parallel-core GPU works in tandem with multi-core CPUs:
  - as a purpose-built device able to assist in performing complex rendering calculations
  - as a separated dedicated graphics resources including graphics processor (Multiple ALUs, Arithmetic Logic Units) and memory (Fast onboard memory)
  - as a co-processor to accelerate through processing of large blocks of data and portions of the code

From a user's perspective, the application runs faster because it's using the massively parallel processing power of the GPU to boost performance. This is known as "heterogeneous" or "hybrid" computing.

- Unlike the threads and processes of UPC/OpenMP and MPI, CUDA adds extra layers of organization of the threads: programmers set up *blocks* containing a certain number of threads, and organize the blocks into a *grid*.

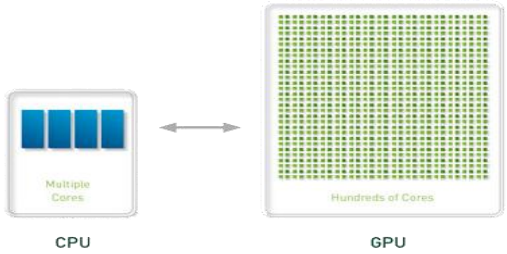


https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

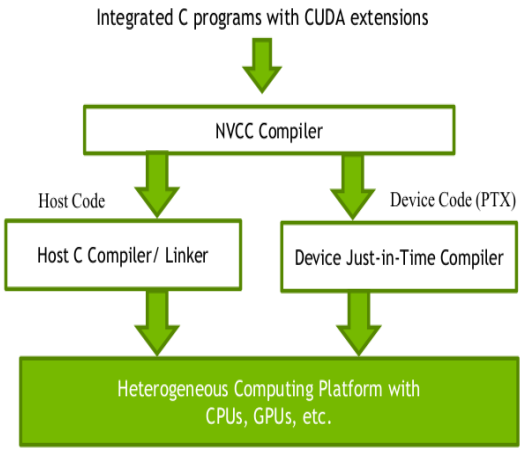
e.g., RTX 3080 has 8,704 cores, 68 SMs, with 128 CUDA cores per SM.

- CUDA organizes threads in a **hierarchical** structure:
  - **Threads**: Basic units of execution.
  - **Thread Blocks**: Groups of threads that execute together and share memory.
  - **Grids**: Arrays of thread blocks, allowing for massive parallelism.

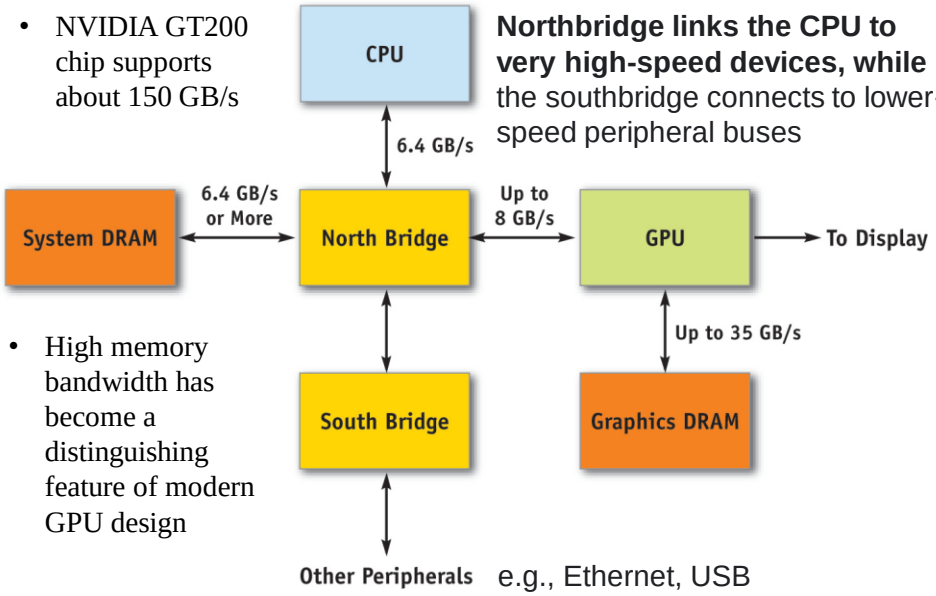
- CUDA provides various memory spaces optimized for different tasks:
  - **Global Memory**: Accessible by all threads, but slower.
  - **Shared Memory**: Faster, block-local memory shared by threads within a block.
  - **Registers**: Private, fast memory for individual threads.







- NVIDIA GT200 chip supports about 150 GB/s



Northbridge links the CPU to very high-speed devices, while the southbridge connects to lower-speed peripheral buses

- High memory bandwidth has become a distinguishing feature of modern GPU design

ıe Overall System Architecture of a PC, Source: from GPU Gems 2 (NVIDIA), the architecture of the GeForce 6 SeriesGPUs.

What is GPGPU ?

- General-Purpose computing on a Graphics Processing Unit
- Using graphic hardware for non-graphic computations
- Typical Applications:
  - Machine Learning (e.g., training deep neural networks).
  - Scientific Computing (e.g., simulations, molecular dynamics).
  - Image and Signal Processing (e.g., medical imaging, computer vision).
  - Cryptography (e.g., blockchain mining).
  - Financial Modeling (e.g., Monte Carlo simulations).

What is GPGPU ?

- Some researchers (and Sony and NVIDIA) took notice of the raw performance of GPUs to solve compute-intensive science and engineering problems

- To access the computational resources, a programmer has to map the original problem to the GPU architecture, i.e., to run many simultaneous instances of a compute function

What is GPGPU ?

- Think of the GPU like a processor.
- NVIDIA developed the CUDA C/C++ compiler, libraries, and runtime software
- TESLA Architecture

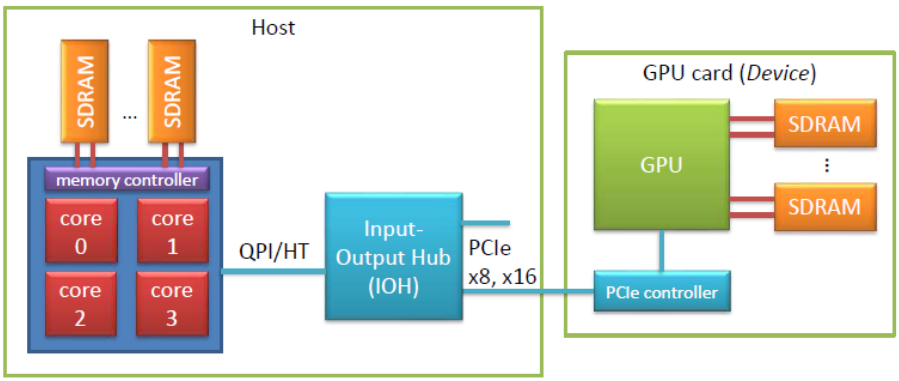
- GPU Computing Tesla C870 transforme un système standard en un supercalculateur avec plus de 500 gigaflops, 128 processeurs, un environnement de développement en langage C pour le GPU, une suite d'outils pour développeurs.

- C870 peut être utilisé conjointement avec des systèmes CPU multi-coeur pour créer une solution informatique flexible qui s'intègre parfaitement dans votre infrastructure informatique.

Outline

- GPU Computing
- **CUDA C programming**

General-purpose GPU computing



- CUDA is a parallel computing platform and programming model created by NVIDIA and implemented by the graphics processing units (GPUs) that they produce

The CUDA programming model

- the CUDA threads execute on a physically separate device that operates as a coprocessor to the host running the C program :
  - the kernels execute on a GPU
  - and the rest of the C program executes on a CPU.

- both the host and the device maintain their own separate memory spaces in DRAM, referred to as host memory and device memory, respectively.
  - a program manages the global, constant, and texture memory spaces visible to kernels through calls to the CUDA runtime (described in Programming Interface).
  - This includes device memory allocation and deallocation as well as data transfer between host and device memory.

Introduction to CUDA C

- What will you learn?
  - Start from “Hello, World!”
  - Write and launch CUDA C kernels
  - Manage GPU memory
  - Run parallel kernels in CUDA C
  - Parallel communication and synchronization
  - Race conditions and atomic operations

CUDA C

CUDA C extends standard C as follows

– Function type qualifiers to specify whether a function executes on the host or on the device

– Variable type qualifiers to specify the memory location on the device

– A new directive to specify how a kernel is executed on the device

– Four built-in variables that specify the grid and block dimensions and the block and thread indices

Function Type Qualifiers

		Executed on the	Only callable from the
`__device__`	`float DeviceFunc()`	device	device
`__global__`	`void KernelFunc()`	device	host
`__host__`	`float HostFunc()`	host	host

- Host : The CPU and its memory (host memory)
- Device : The GPU and its memory (device memory)

Hello, World!

```
#include <stdlib.h>
#include <stdio.h>
int main(){
    printf("Hello World!\n");
    return EXIT_SUCCESS;
}
```

- This basic program is just standard C that runs on the host
- NVIDIA's compiler (nvcc) will not complain about CUDA programs with no device code
- At its simplest, CUDA C is just C!

Hello, World! with Device Code

```
#include <stdlib.h>
#include <stdio.h>

__global__ void kernel( void ) { }

int main(){
    kernel<<< 1, 1 >>>();
    printf("Hello World!\n");
    return EXIT_SUCCESS;
}
```

Hello, World! with Device Code

```
__global__ void kernel( void ) { }
```

- __global__ indicates that the function
  - Runs on the device
  - Called from host code

- nvcc splits source file into host and device components

NVIDIA's compiler handles device functions like kernel()
Standard host compiler handles host functions like main() gcc
Microsoft Visual C

Hello, World! with Device Code

```
int main( void ) {
    kernel<<< 1, 1 >>>();
    printf( "Hello, World!\n" );
    return 0;
}
```

- Triple angle brackets mark a call from host code to device code

- This is all that's required to execute a function on the GPU!

- We'll discuss the parameters inside the angle brackets later

A More Complex Example

- A simple kernel to add two integers:

```
__global__ void add ( int a, int b, int *c) {
    *c = a + b;
}
```

  - add() runs on the device ; a, b, and c must point to device memory
  - How do we allocate memory on the GPU?

The arguments **a** and **b** are passed by value. In CUDA kernels, parameters that are meant to represent arrays or data structures residing in GPU memory should be passed as pointers.

Memory Management

- Host and device memory are distinct entities
  - Device pointers point to GPU memory
  - Host pointers point to CPU memory
- Basic CUDA API for dealing with device memory
  - cudaMalloc(), cudaFree(), cudaMemcpy()
  - Similar to their C equivalents, malloc(), free(), memcpy()

Memory Management

```
int main( void ) {
    int a, b, c;                        // host copies of a, b, c
    int *dev_a, *dev_b, *dev_c;  // device copies of a, b, c
    int size = sizeof( int );

cudaMalloc ( ( void**)&dev_a, size);
cudaMalloc(  (void**)&dev_b, size );
cudaMalloc ( (void**)&dev_c, size );
```

A More Complex Example: main()

```
cudaMemcpy(dev_a, &a, size, cudaMemcpyHostToDevice);
cudaMemcpy(dev_b, &b, size, cudaMemcpyHostToDevice);
cudaMemcpy(dev_c, &c, size, cudaMemcpyHostToDevice);

 // launch add() kernel on GPU, passing parameters
add<<< 1, 1 >>>( dev_a, dev_b, dev_c );
// copy device result back to host copy of c
cudaMemcpy( &c, dev_c, size, cudaMemcpyDeviceToHost );

 cudaFree(dev_a);
 cudaFree(dev_b);
 cudaFree(dev_c);

return 0;
```

- <<< 1, 1 >>> Specifies the number of blocks and threads per block for the kernel launch
- Executed with a **single block**, and that block will contain **only one thread**

Parallel Programming in CUDA C

- So how do we run code in parallel on the device?
  - via the parameters between the triple angle brackets:

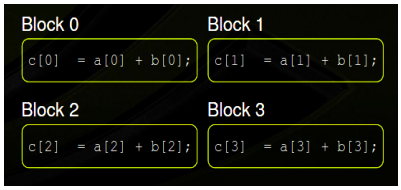
```
add<<< N, 1 >>>( dev_a, dev_b, dev_c );
```

- Instead of executing add() once, add() executed N times in parallel. Launch N copies of add() with add<<<N,1>>>(...)

But wait…GPU computing is about massive parallelism

Parallel Addition: add()

- Terminology (**vecadd0.cu**)
  - Each parallel invocation of add() referred to as a block
  - Kernel can refer to its block's index with the variable blockIdx.x
  - Each block adds a value from a[] and b[], storing the result in c[]:




```
__global__ void add( int *a, int *b, int *c ) {
        c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
```

By using **blockIdx.x** to index arrays, each block handles different indices

Parallel Addition: main()

```
#define N 512
int main( void ) {
    // host copies of a, b, c
    Int* a, b, c;
    // device copies of a, b, c
    int *dev_a, *dev_b, *dev_c; int size = sizeof( int );

    //allocate device copies of a, b , c
    cudaMalloc ( ( void**)&dev_a, size);
    cudaMalloc(  (void**)&dev_b, size );
    cudaMalloc ( (void**)&dev_c, size );
```

Parallel Addition: main() (cont)

```
    a=(int *) malloc (size);
    b=(int *) malloc (size);
    c=(int *) malloc (size);

    //copy inputs to device
    cudaMemcpy(dev_a, &a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(dev_b, &b, size, cudaMemcpyHostToDevice);
    cudaMemcpy(dev_c, &c, size, cudaMemcpyHostToDevice);

    // launch add() kernel on GPU with N parallel blocks. Several
    blocks with one thread each.
    add<<< N, 1 >>>( dev_a, dev_b, dev_c );
```

Parallel Addition: main() (cont)

```
    // copy device result back to host copy of c
    cudaMemcpy( &c, dev_c, size, cudaMemcpyDeviceToHost );

    cudaFree(dev_a);
    cudaFree(dev_b);
    cudaFree(dev_c);

    Free(a); free(b); free (c);

    return 0;
}
```

Threads

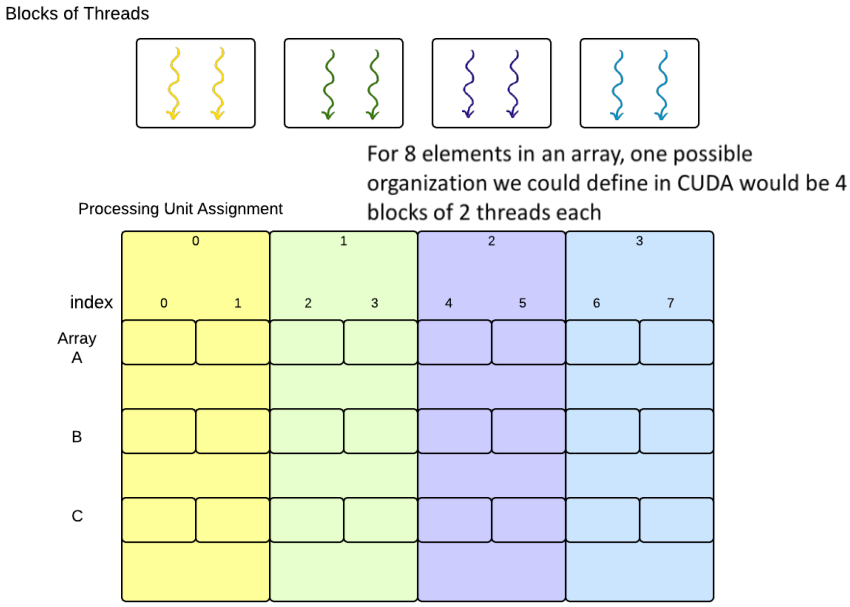
- Terminology: A block can be split into parallel threads (**vecadd1.cu**)
- Let's change vector addition to use parallel threads instead of parallel blocks:
- We use *threadIdx.x* instead of *blockIdx.x* in add()

```
__global__ void add ( int       *a, int *b, int *c ) {
    c[threadIdx.x] = a[threadIdx.x ] + b[threadIdx.x ];
}
```

- **One block with several threads**

Threads

http://selkie.macalester.edu/



Threads

http://selkie.macalester.edu/

```
#include <stdio.h>
#include <cuda_runtime.h>
 int main() {
   cudaDeviceProp prop;
   cudaGetDeviceProperties(&prop, 0); // 0 for device 0 (or specify
device number)
    printf("Max Threads per Block: %d\n", prop.maxThreadsPerBlock);

printf("Max Grid Size (X, Y, Z): (%d, %d, %d)\n", prop.maxGridSize[0],
prop.maxGridSize[1], prop.maxGridSize[2]);
    return 0;
}
```

- On most modern NVIDIA GPUs, the maximum number of threads per block is 1024
- The maximum number of blocks in each dimension can be around 2^31 - 1 (2147483647) for modern GPUs.

Threads

- If we have M threads/block, a unique array index for each entry is given by

```
int index = threadIdx.x + blockIdx.x * M;

int index =       x      +      y      * width;
```

- **threadIdx.x** represents a thread's index along the x dimension within the block.
- **blockIdx.x** represents a thread's block's index along the x dimension within the grid.
- **blockDim.x** represents the number of threads per block in the x direction.

Addition with Threads and Blocks

- The blockDim.x is a built-in variable for threads per block:

```
int index= threadIdx.x + blockIdx.x * blockDim.x;
```

- A combined version of the vector addition kernel to use blocks and threads:

```
__global__  void add( int *a, int *b, int *c) {
        int index = threadIdx.x + blockIdx.x * blockDim.x;
        c[index] = a[index] + b[index];
}
```



Addition with Threads and Blocks



add<<< N/THREADS_PER_BLOCK, THREADS_PER_BLOCK>>>

If you have N elements to process and THREADS_PER_BLOCK threads per block, the execution configuration << N/THREADS_PER_BLOCK, THREADS_PER_BLOCK >>> would launch enough blocks to cover all N elements with THREADS_PER_BLOCK threads in each block.

N/THREADS_PER_BLOCK blocks and THREADS_PER_BLOCK threads gave us N threads tot

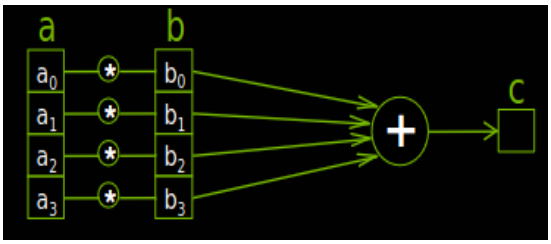
www.nvidia.com/

Overall structure of a CUDA program

- Define the code, which will run on the device in a separate function, called the **kernel function**.
- In the main program running on the host's CPU:
  - **allocate memory** on the host for the data arrays.
  - **initialze the data arrays** in the host's memory.
  - **allocate separate memory** on the GPU device for the data arrays.
  - **copy data arrays** from the host memory to the GPU device memory.
- On the GPU device, **execute the kernel function** while specifying how many blocks and threads per block to use.
- **Copy the computed values** from the GPU device memory back to the host's memory.

Dot Product

- Unlike vector addition, dot product is a reduction from vectors to a scalar



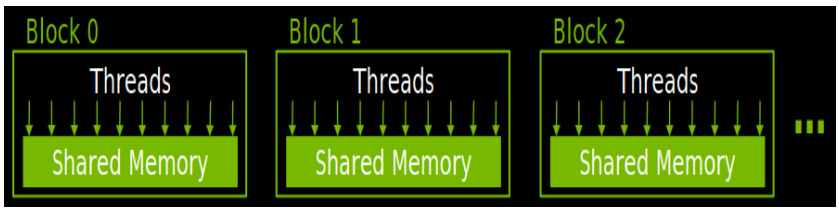
Dot Product

```
__global__ void dot (int *a, int *b, int *c ) {
    // Each thread computes a pairwise product
    int temp = a[threadIdx.x] * b[threadIdx.x];
}
```

How to compute the final sum ?

Sharing Data Between Threads

- Terminology:
  - There is a shared memory for threads in a block
  - Declared with the __shared__ CUDA keyword
  - Extremely fast, on-chip memory
  - Not visible to threads in other blocks running in parallel



Synchronization

- We can synchronize threads with the function __syncthreads()
- Threads are only synchronized within a block

Synchronization

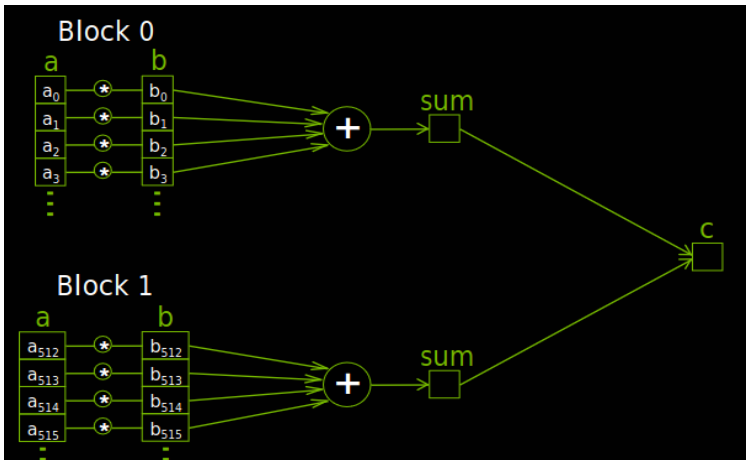
```
__global__ void dot( int *a, int *b, int *c ) {
    __shared__ int temp[N];
    temp[threadIdx.x]= a[threadIdx.x] * b[threadIdx.x];
    __syncthreads();
        if( 0== threadIdx.x ) {
            int sum = 0;
            for( int i =       0; i        < N; i++ ) sum += temp[i];
            *c = sum;
        }
}
```

- // launch dot() kernel with 1 block and N threads

```
dot<<<   1, N>>> ( dev_a, dev_b, dev_c );
```

Multiblock Dot Product

- Launching with one block will not utilize much of the GPU



Multiblock Dot Product

```
__global__ void dot( int *a, int *b, int *c ) {
    __shared__ int temp[THREADS_PER_BLOCK];
    temp[threadIdx.x]= a[threadIdx.x] * b[threadIdx.x];
    __syncthreads();
        if( 0== threadIdx.x ) {
            int sum = 0;
            for( int i =0; i<THREADS_PER_BLOCK; i++ ) sum +=
            temp[i];
            *c = sum;
        }
}
dot<<< N/THREADS_PER_BLOCK, THREADS_PER_BLOCK>>>
```

Multiblock Dot Product

- To avoid race conditions, many atomic operations on memory available with CUDA C
- What actually takes place to execute the line in question: *c += sum;
- What if two threads are trying to do this at the same time?
- Atomics ensure correct results when multiple threads modify memory

atomicAdd()	atomicInc()
atomicSub()	atomicDec()
atomicMin()	atomicExch()
atomicMax()	atomicCAS()

Cuda, MPI

- CUDA (GPU Programming)
  - **Cores** are GPU cores, the smallest computational units that execute individual instructions. A core typically executes one thread at a time.
  - A **thread** is the basic unit of execution in CUDA. Threads execute kernel functions in parallel on GPU cores.
  - **Task** often maps to the work assigned to a thread or group of threads, such as processing a portion of a dataset.
  - **Process** is a host-level concept (launched as a single process on the host CPU). Processes manage and launch GPU kernels and coordinate data transfer between the CPU and GPU .

Cuda, MPI

- MPI (Distributed Memory Programming)
  - **Cores** refers to a CPU core in the hardware where MPI processes run. Each process is typically mapped to one core.
  - MPI is process-oriented, but with MPI + OpenMP (hybrid model), **threads** can be created within a single MPI process. Threads share memory within the process.
  - A **task** in MPI corresponds to a process performing a chunk of the overall workload. Each MPI process is an independent task.
  - MPI **process** runs independently, has its own memory space, and communicates with other processes through message passing

Cuda, MPI

- UPC Programming model
  - A **core** is a CPU core where UPC threads execute.
  - A **thread** is an independent unit of execution. UPC creates a fixed number of threads at the start of the program, each with its own private memory space and access to a global shared memory.
  - A **task** in UPC corresponds to the work assigned to a thread, often involving access to shared memory.
  - UPC is t**hread-based**, not process-based, but each thread acts like an independent process when accessing private memory. Communication between threads happens implicitly through the global address space.

Cuda, MPI

- Key Relationships
  - In CUDA, **threads** map directly to **GPU cores**, and tasks are granular (thread-level). Ideal for **fine-grained**, massively parallel workloads on GPUs.
  - In MPI, **processes** are the primary parallel abstraction, and **tasks** are equivalent to processes. Excels in **distributed memory systems** with **coarse-grained** parallelism.
  - In UPC, **threads** are used instead of **processes**, sharing a global address space for communication. Bridges shared and distributed memory models, useful for **moderate parallelism** with **global memory sharing**.

Cuda, MPI

Aspect	CUDA (GPU)	MPI (Distributed)
Core	Executes threads	Executes MPI processes
Thread	Basic unit of execution	Optional (via OpenMP, etc.)
Task	GPU kernel operations	Work assigned to processes
Process	Host-side control	Independent, explicit memory
Memory Model	Shared, hierarchical	Distributed (message passing)